Cycles de mesures et régulation de température ambiante

via chaîne de mesure avec Arduino & PID

JALAL MANSOUR Imrane

28898

Sommaire

1 Introduction

2 Problématique

3 Étude de capteur de température

4 Boucle de régulation

5 Modélisation thermique

Sommaire

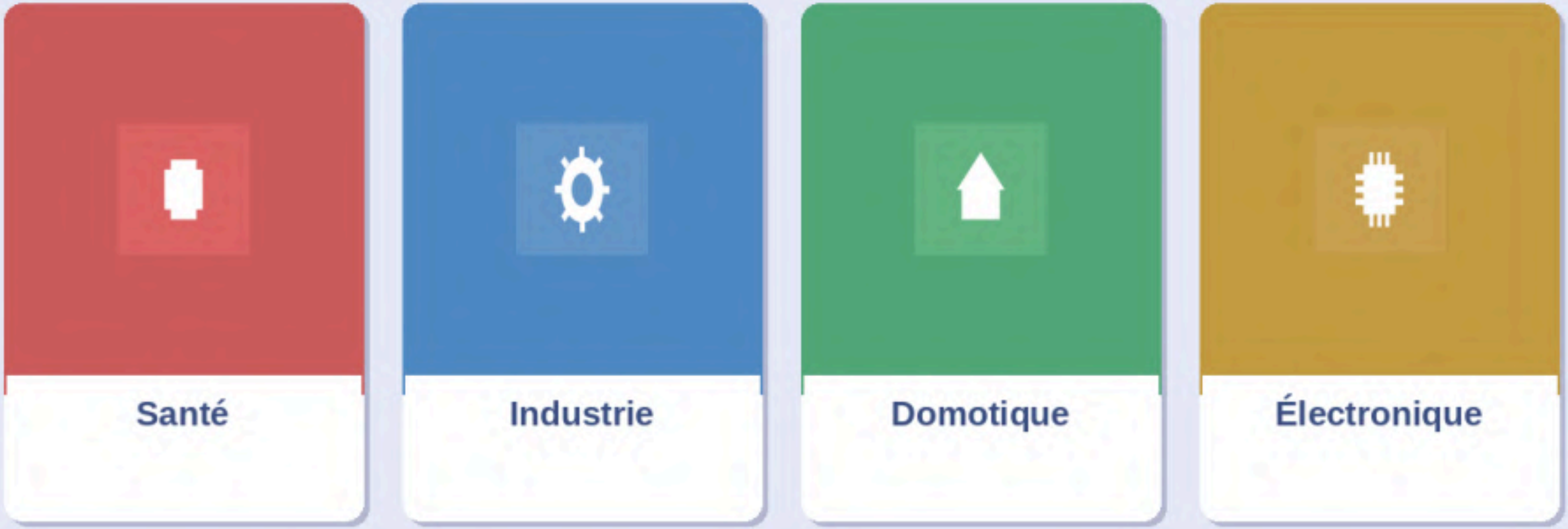
1 Introduction

2 Problématique

3 Étude de capteur de température

4 Boucle de régulation

5 Modélisation thermique


Santé
Industrie
Domotique
Électronique

Sommaire

1 Introduction

2 Problématique

3 Étude de capteur de température

4 Boucle de régulation

5 Modélisation thermique

Problématique :

Comment réguler la température ambiante
en **boucle fermée** via **Arduino & PID** ?

Sommaire

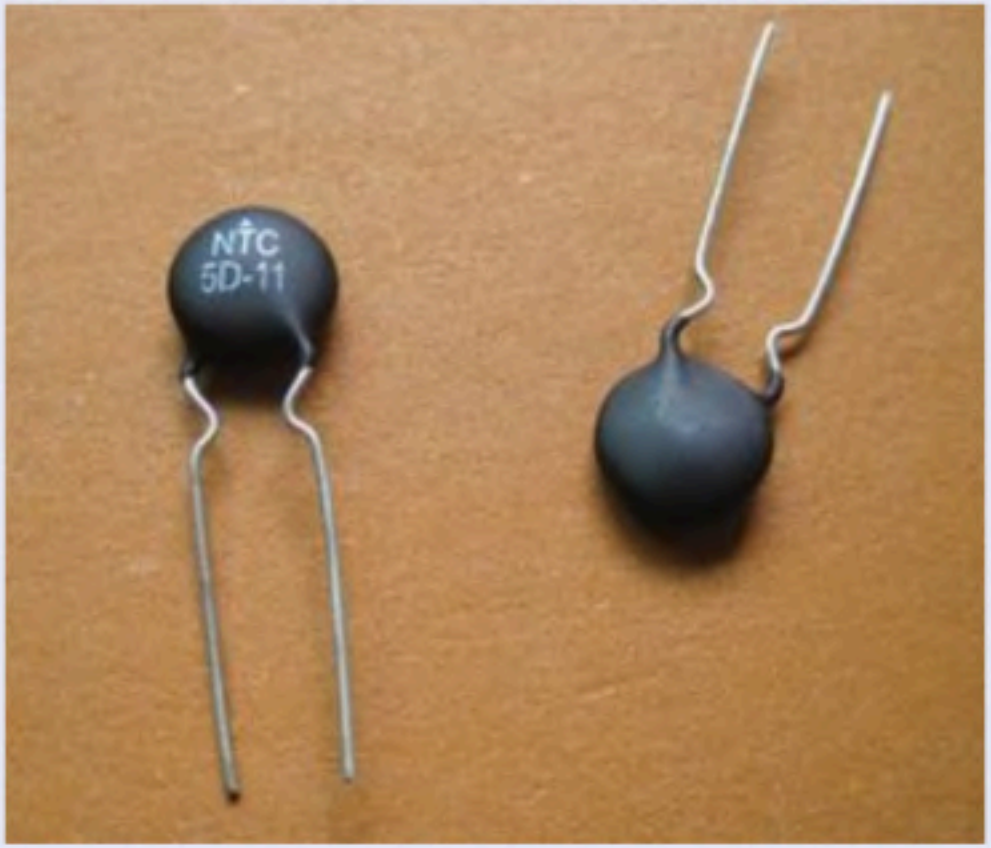
1 Introduction

2 Problématique

3 Étude de capteur de température

4 Boucle de régulation

5 Modélisation thermique



La CTN est un composant dont la résistance varie en fonction de la température.

La relation entre résistance et température est donnée par la **loi de Boltzmann** :

$$R(T) = R_0 \exp\left[B\left(\frac{1}{T} - \frac{1}{T_0}\right)\right]$$

Paramètres de la formule :

- T : température à mesurer, exprimée en **Kelvin**
- $T_0 = 298{,}15\,\mathrm{K}$: température de référence ($= 25\,^\circ\mathrm{C}$)
- R_0 : résistance de la CTN mesurée exactement à T_0 (donnée fabricant, ex. $10\,\mathrm{k}\Omega$)
- $B \approx 3950\,\mathrm{K}$: constante thermique caractéristique du matériau

- $\mathbf{T_0 = 25\ °C}$ — *température de référence standard*
- $\mathbf{R_0}$ — *résistance mesurée à T_0 (donnée fabricant)*

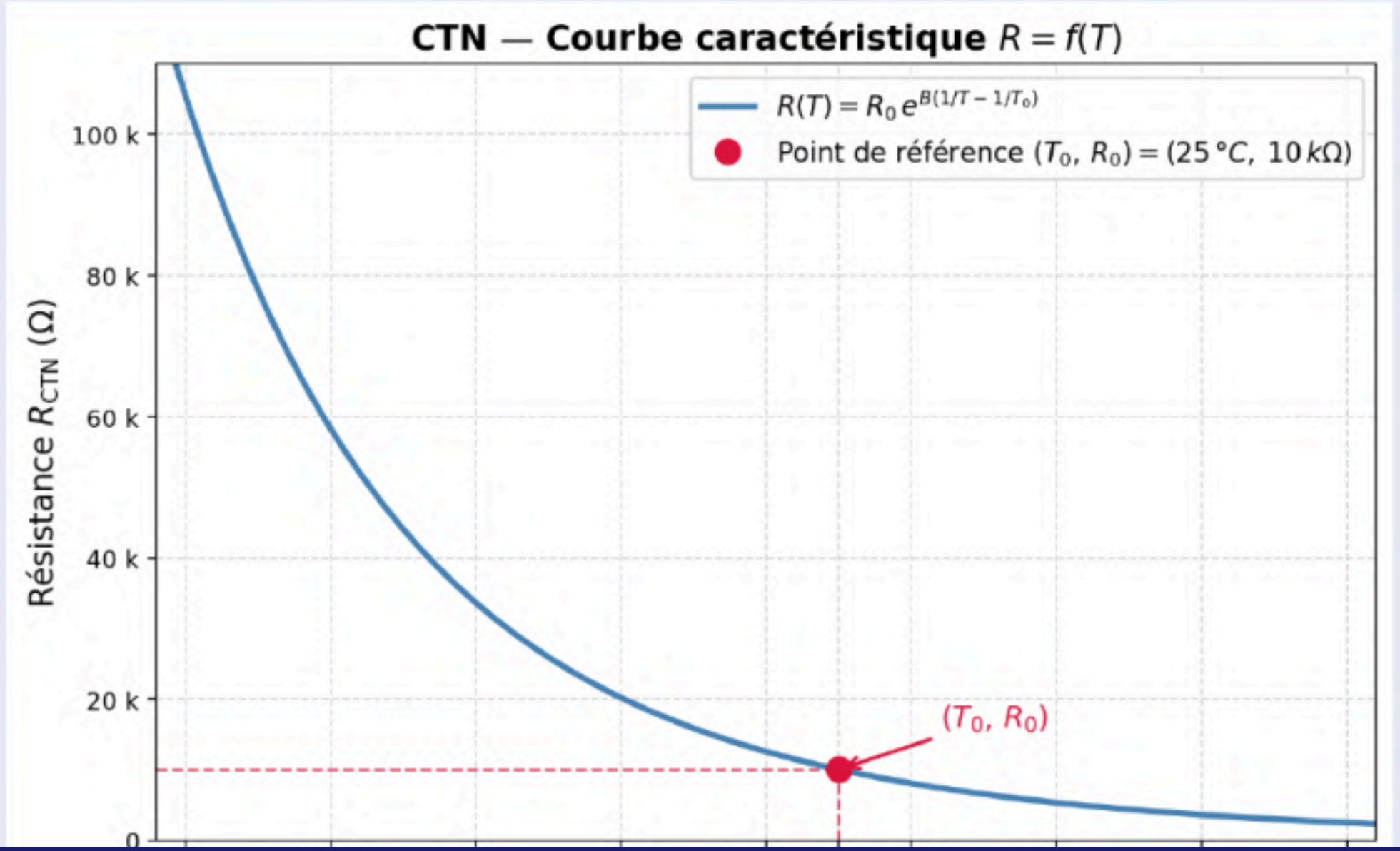
$$\mathbf{R(T_0) = R_0 \cdot e^{\,} = R_0}$$

(T_0, R_0) est le point d'ancrage de la courbe $R(T)$

On mesure $(\mathbf{R_1, T_1})$ et $(\mathbf{R_2, T_2})$, puis on isole B :

$$\mathbf{B} = \frac{ln(R_1/R_2)}{1/T_1 - 1/T_2}$$

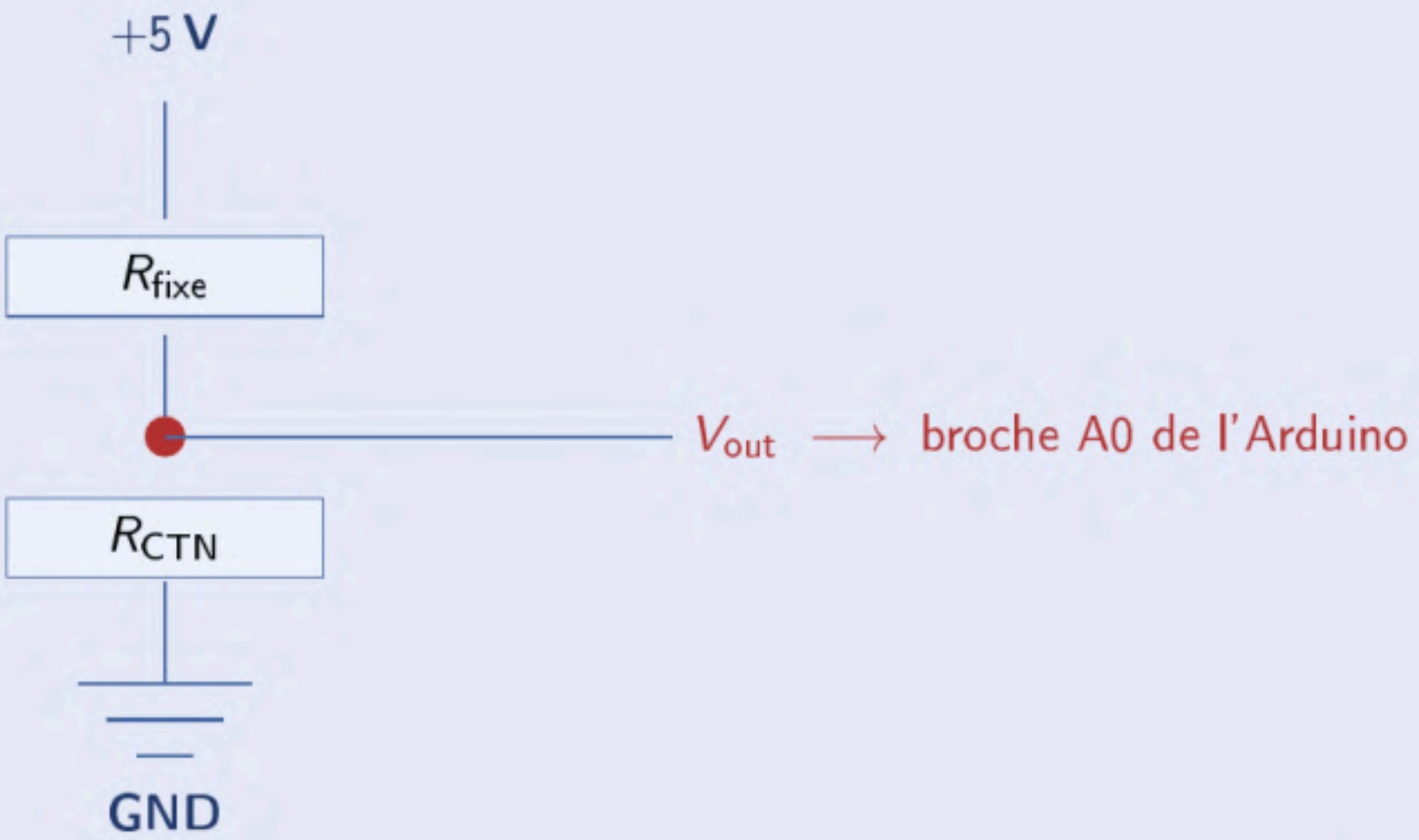
Dans notre cas : **B = 3950 K**


CTN — Courbe caractéristique $R = f(T)$
$R(T) = R_0\, e^{B(1/T - 1/T_0)}$
Point de référence $(T_0,\ R_0) = (25\,°C,\ 10\,k\Omega)$
Résistance R_{CTN} (Ω)
100 k
80 k
60 k
40 k
20 k
0
$(T_0,\ R_0)$

Problème : L'Arduino ne mesure pas directement une résistance.

Solution : Un diviseur de tension convertit R en tension lisible.





Les deux résistances R_{fixe} et R_{CTN} sont montées **en série** entre +5 V et GND. V_{out} est

$$V_{cc} - V_{\text{Rfixe}} - V_{\text{RCTN}} = 0$$

$$V_{cc} = I \times (R_{\text{fixe}} + R_{\text{CTN}})$$

$$\text{I} = \frac{V_{cc}}{R_{\text{fixe}} + R_{\text{CTN}}}$$

$$V_{out} = V_{cc} \times \frac{R_{\text{CTN}}}{R_{\text{fixe}} + R_{\text{CTN}}}$$

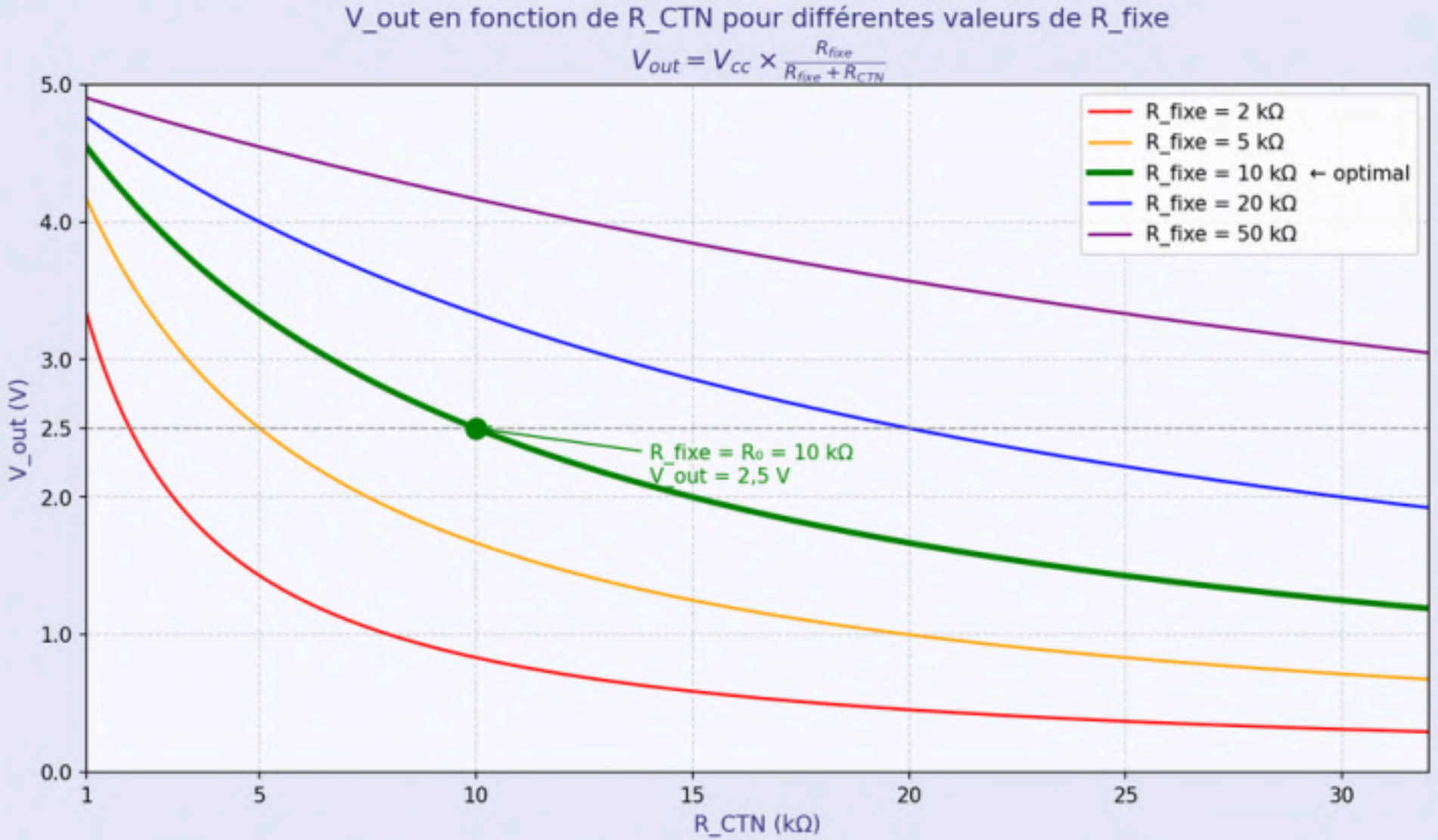
$$V_{out} = V_{cc} \times \frac{R_{\text{CTN}}}{R_{\text{fixe}} + R_{\text{CTN}}}$$

$$V_{out} \times (R_{\text{fixe}} + R_{\text{CTN}}) = V_{cc} \times R_{\text{CTN}}$$

$$V_{out} \times R_{\text{fixe}} = R_{\text{CTN}} \times (V_{cc} - V_{out})$$

$$R_{\text{CTN}} = R_{\text{fixe}} \times \frac{V_{cc} - V_{out}}{V_{out}}$$

V_out dépend de R_CTN et R_fixe — choisir R_fixe = R_0 = 10 kΩ centre V_out à 2,5 V.



Conversion R → V

L'Arduino mesure uniquement une tension entre 0 V et 5 V.

R_fixe est indispensable pour convertir la variation de R_CTN en une tension lisible par l'ADC.

Étalonnage de la plage

La valeur de R_fixe détermine comment V_out occupe [0 V ; 5 V].

Choisir R_fixe = R_0 = 10 kΩ :

→ centre V_out à 2,5 V à 25 °C

→ maximise la résolution ADC

CAN 10 bits → 2^{10} = 1024 niveaux → $\Delta V \approx 4{,}9$ mV

$$\text{val} = \frac{V_{out} \times 1023}{5}$$

Correspondance :

V_out	val
0 V	0
2,5 V	512
5 V	1023

Inverse — reconvertir val en tension :

$$V_{out} = \frac{\text{val} \times V_{cc}}{1023}$$

Résolution : 1 niveau = 4,9 mV → précision suffisante pour mesurer la température

Étape 1 — Reconversion de la valeur ADC en tension

L'Arduino lit une valeur numérique `val` entre 0 et 1023. On la reconvertit en tension :

$$V_{\text{out}} = \frac{\texttt{val} \times V_{cc}}{1023}$$

Étape 2 — Calcul de la résistance de la CTN

On utilise la formule du diviseur de tension inversée (démontrée précédemment) :

$$R_{\text{CTN}} = R_0 \cdot \frac{V_{cc} - V_{\text{out}}}{V_{\text{out}}}$$

$$R_{CTN} = R_0 \cdot e^{B(1/T - 1/T_0)}$$

$$ln\left(\frac{R_{CTN}}{R_0}\right) = B \cdot \left(\frac{1}{T} - \frac{1}{T_0}\right)$$

$$\frac{1}{T} = \frac{1}{T_0} + \frac{1}{B} \cdot \ln\left(\frac{R_{CTN}}{R_0}\right)$$

$$\mathbf{T}_{Kelvin} = \frac{1}{\frac{1}{T_0} + \frac{1}{B} \cdot \ln\left(\frac{R_{CTN}}{R_0}\right)}$$

$$T_{°C} = T_{Kelvin} - 273{,}15$$

$$\frac{R_{CTN}}{R_0} = \frac{V_{cc} - V_{out}}{V_{out}}$$

$$\frac{1}{T} = \frac{1}{T_0} + \frac{1}{B} \cdot \ln\left(\frac{V_{cc} - V_{out}}{V_{out}}\right)$$

$$\mathbf{T(°C)} = \frac{1}{1/T_0 + 1/B \cdot \ln(V_{cc} - V_{out})/V_{out}} - 273{,}15$$

$$\frac{V_{cc} - V_{out}}{V_{out}} = \frac{1023 - \text{val}}{\text{val}}$$

$$\frac{1}{T} = \frac{1}{T_0} + \frac{1}{B} \cdot \ln\left(\frac{1023 - \text{val}}{\text{val}}\right)$$

$$\mathbf{T(°C)} = \frac{1}{1/T_0 + 1/\mathrm{B} \cdot \ln(1023 - \text{val})/\text{val}} - 273{,}15$$

V_cc se simplifie — formule indépendante de la tension d'alimentation !

En substituant le résultat précédent, on obtient la **formule ultime** qui relie directement la valeur brute lue par l'Arduino à la température en degrés Celsius :

$$T(C) = \frac{1}{\frac{1}{T_0} + \frac{1}{B}\ln\left(\frac{1023 - \texttt{val}}{\texttt{val}}\right)} - 273{,}15$$

- `val` est la valeur brute ADC lue par `analogRead(A0)`, entre 0 et 1023
- V_{cc} a disparu : la formule est **indépendante** de la tension d'alimentation

Sommaire

1 Introduction

2 Problématique

3 Étude de capteur de température

4 Boucle de régulation

5 Modélisation thermique

À chaque cycle de mesure, on calcule l'**erreur** entre la consigne et la température mesurée :

$$e(t) = T_{\text{consigne}} - T_{\text{mesurée}}(t)$$

Interprétation de l'erreur :

- Si $e(t) > 0$: la température est **trop basse** $\Rightarrow$ le système doit **chauffer**
- Si $e(t) = 0$: la température est **exactement à la consigne** $\Rightarrow$ aucune correction nécessaire
- Si $e(t) < 0$: la température est **trop haute** $\Rightarrow$ le système doit **refroidir**

Le Régulateur PID — Synthèse des 3 Actions

P — Proportionnel

■ **Agit sur :**
Erreur actuelle

✓ **Avantage :**
Réponse rapide

■ **Limite :**
Erreur résiduelle

Corrige immédiatement l'écart actuel

+

I — Intégral

■ **Agit sur :**
Erreur accumulée

✓ **Avantage :**
Supprime l'erreur stat.

■ **Limite :**
Ralentit la réponse

Élimine l'erreur résiduelle sur le long terme

+

D — Dérivé

■ **Agit sur :**
Vitesse de variation

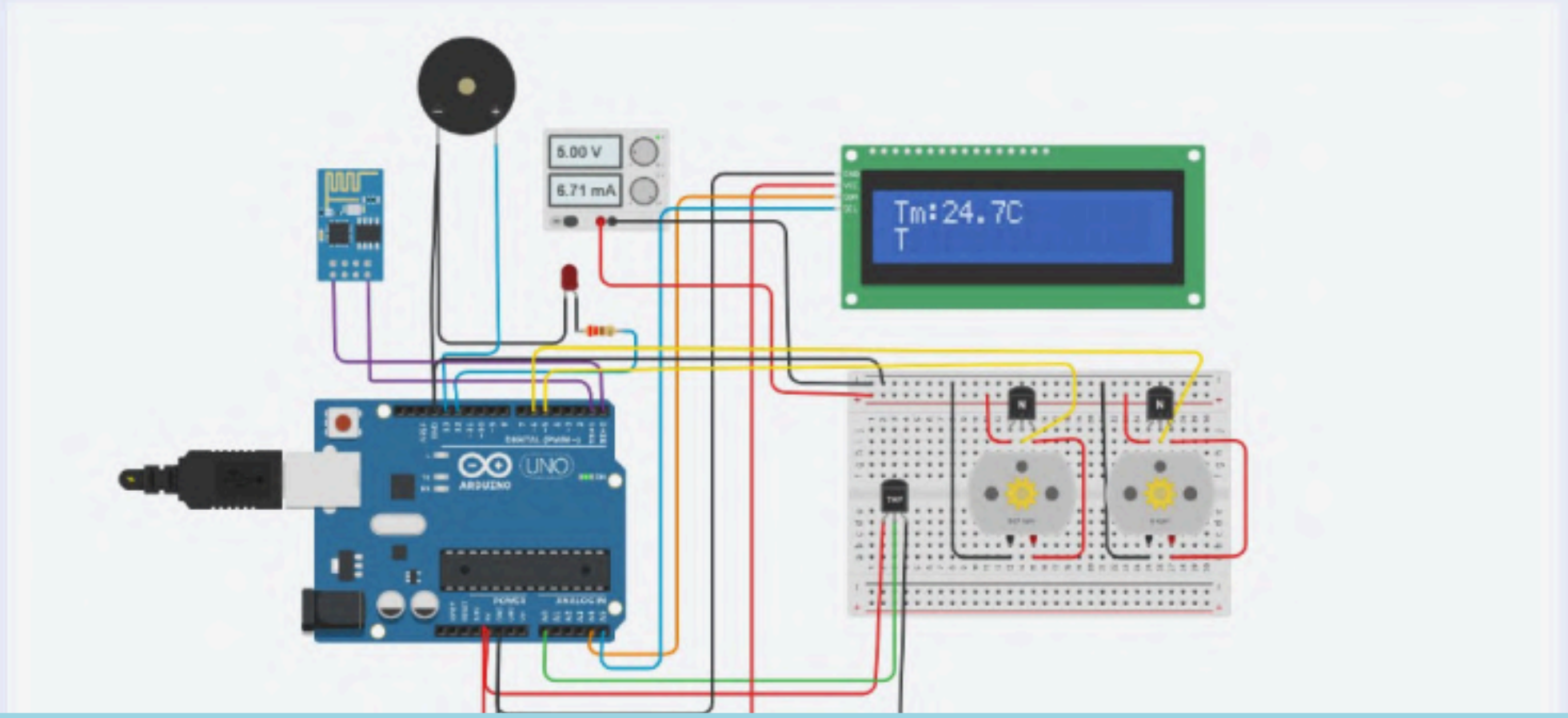
✓ **Avantage :**
Amortit les oscillations

■ **Limite :**
Sensible au bruit

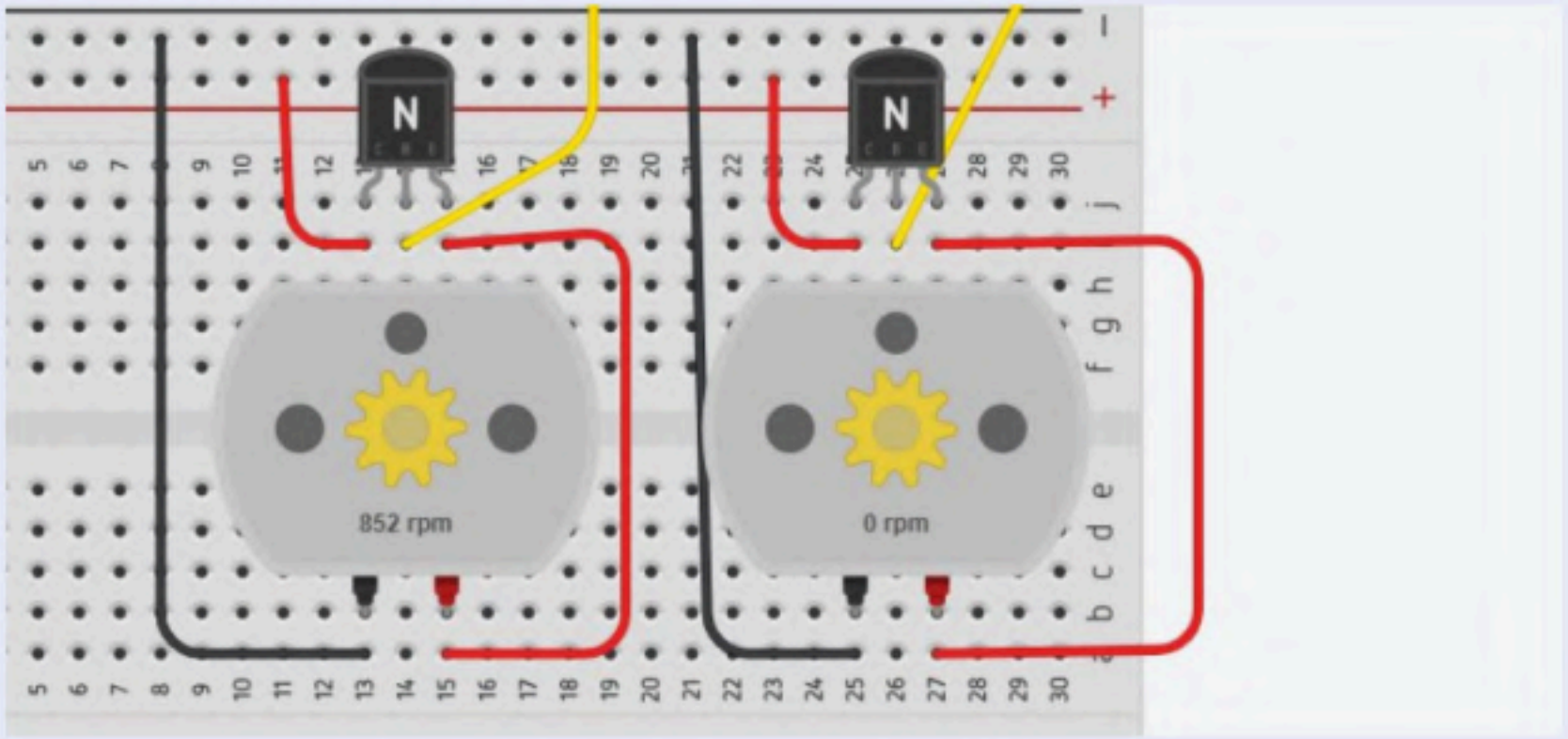
Anticipe les variations futures

P + I + D ⇒ Régulation précise, stable et rapide

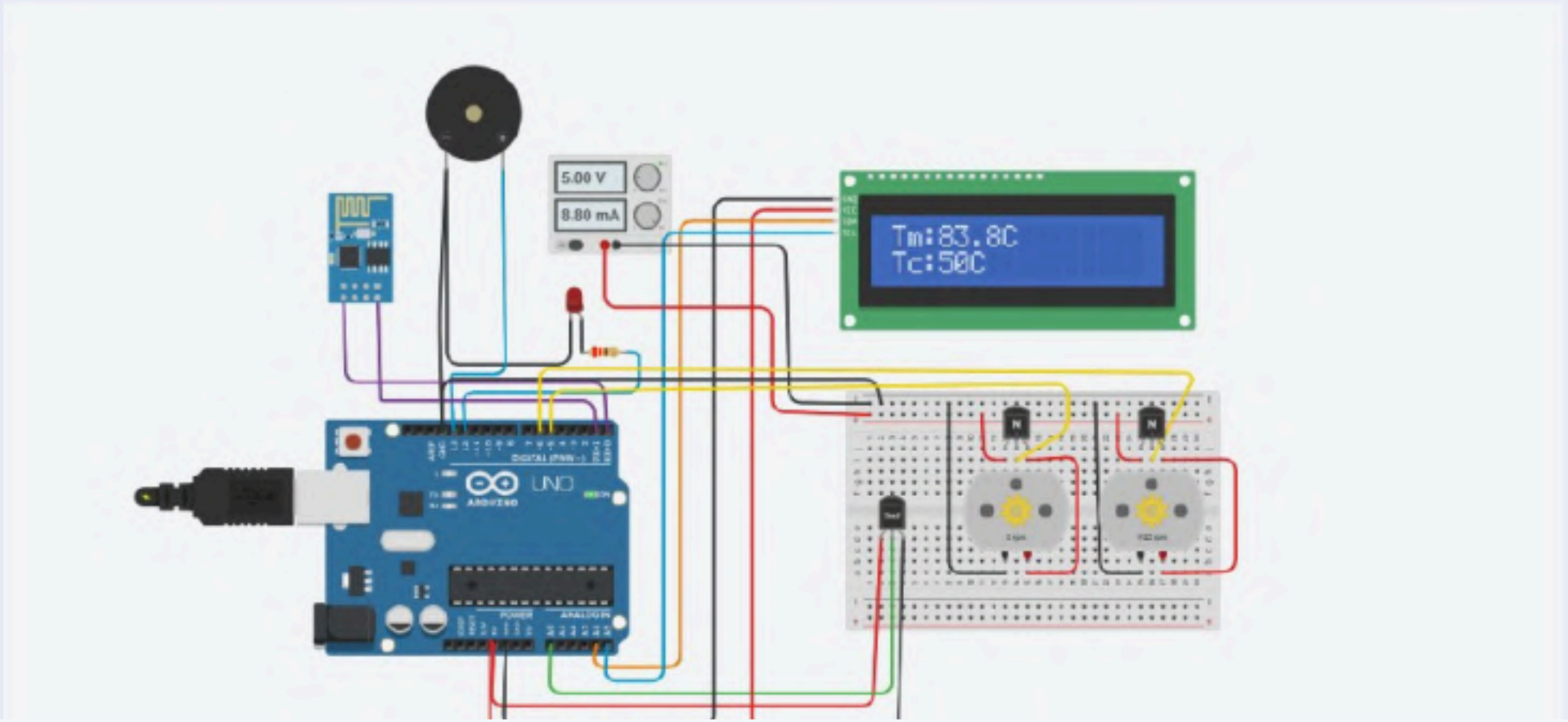
Condition : $T_m = 24{,}7\,°\text{C} < T_c = 50\,°\text{C}$ — **Résultat :** moteur chauffage $= 527$ rpm, moteur refroidissement $= 0$ rpm



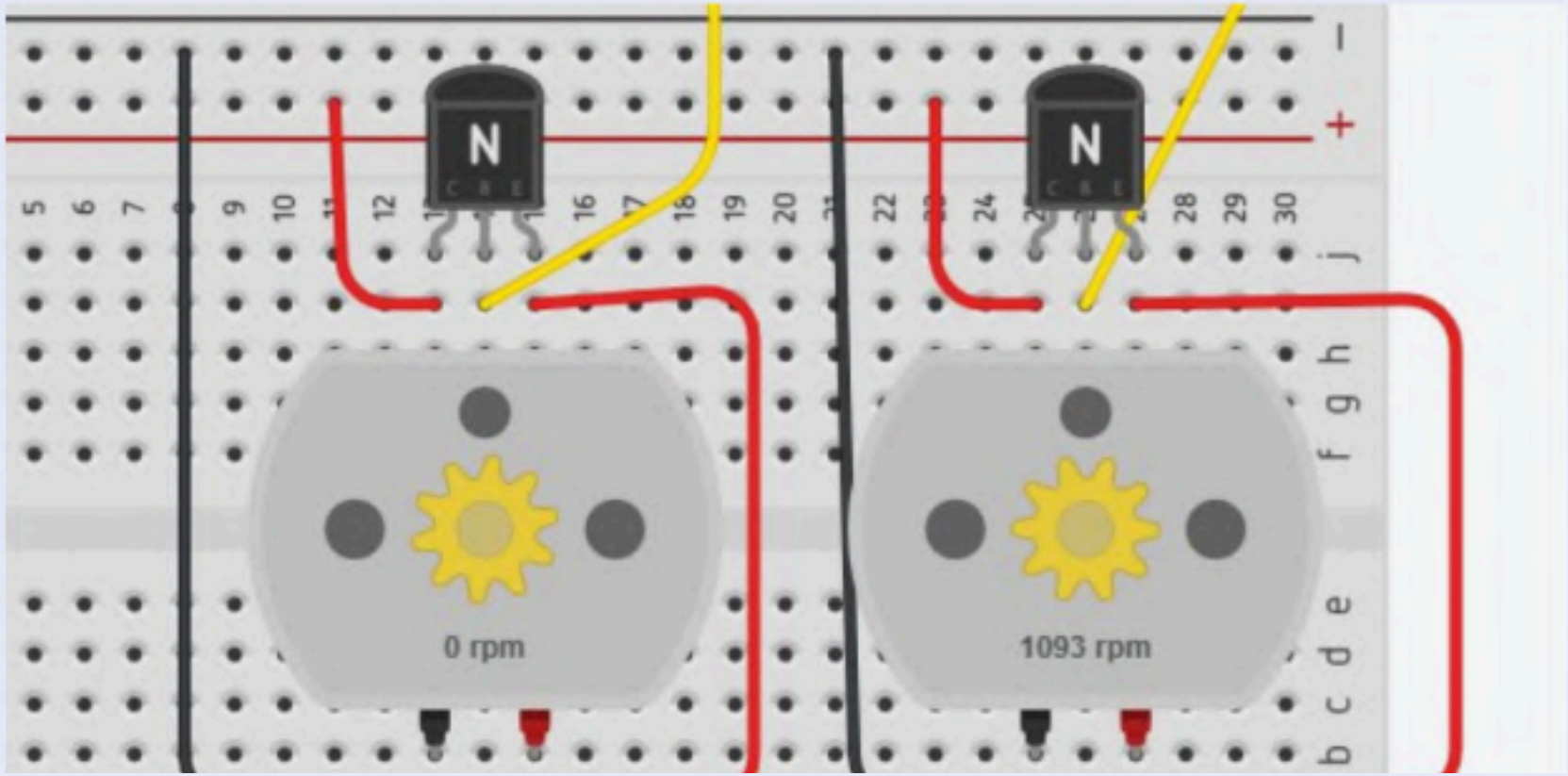
Condition : $T_m \ll T_c$ (température très basse) — **Résultat :** moteur chauffage $= 852$ rpm, moteur refroidissement $= 0$ rpm



Condition : $T_m = 83{,}8\,°C > T_c = 50\,°C$ — **Résultat :** moteur refroidissement $= 722\,\text{rpm}$ + LED alarme allumée



Condition : refroidissement en cours, T_m encore trop haute — **Résultat** : moteur refroidissement $= 1093$ rpm, moteur chauffage $= 0$ rpm



Comportement du régulateur PID selon l'erreur e(t)

CHAUFFAGE

$T_m < T_c \rightarrow e(t) > 0$

Le PID envoie une commande positive au moteur de chauffage

- Plus l'écart est grand, plus le moteur tourne vite

→ L'erreur diminue jusqu'à atteindre la consigne

■ Tm ■ Tc ⇒ Chauffage rapide et puissant

Tm ■ Tc ⇒ Chauffage rapide et puissant

REFROIDISSEMENT

$T_m > T_c \rightarrow e(t) < 0$

Le PID envoie une commande négative au moteur de refroidissement

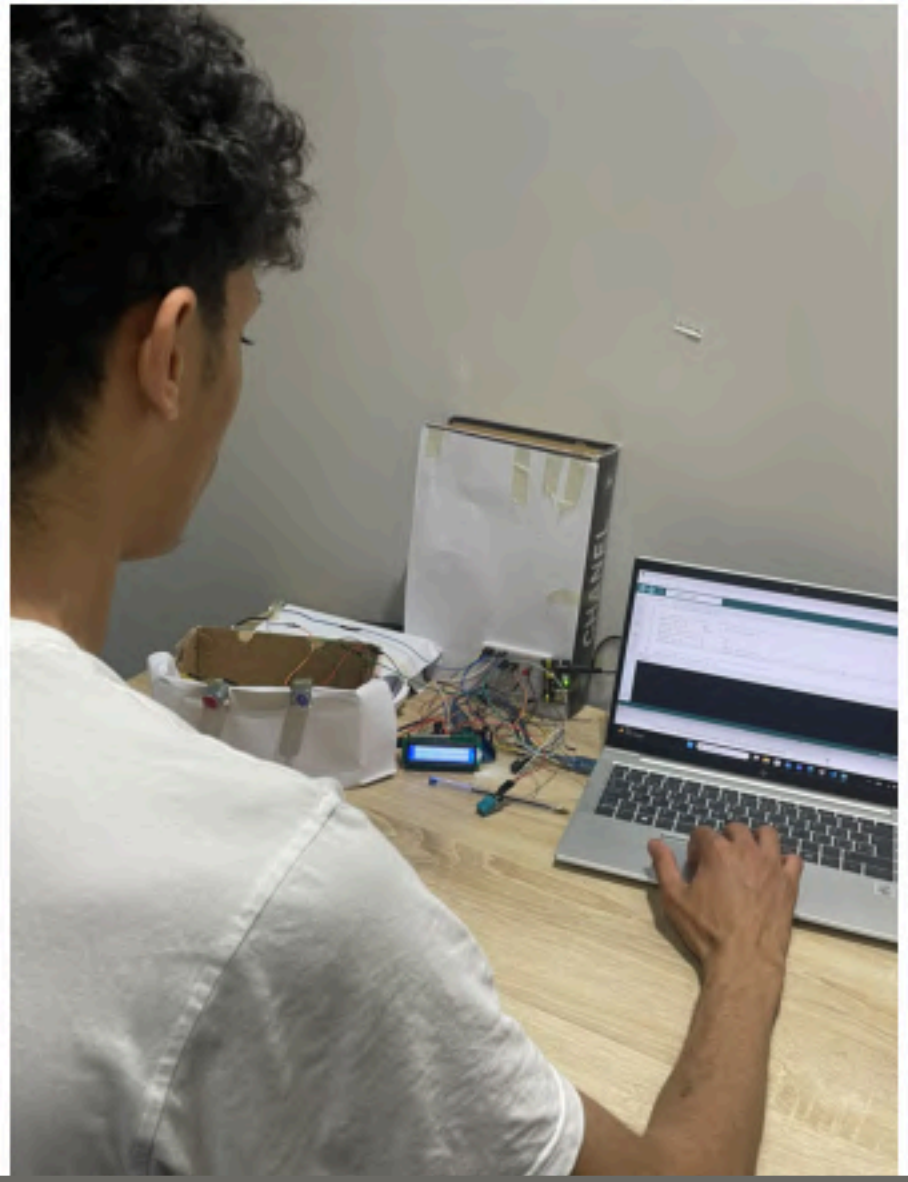
- Plus l'écart est grand, plus le moteur tourne vite

→ L'erreur diminue jusqu'à atteindre la consigne

■ Tm ■ Tc ⇒ Refroidissement rapide et puissant

Tm ■ Tc ⇒ Refroidissement rapide et puissant

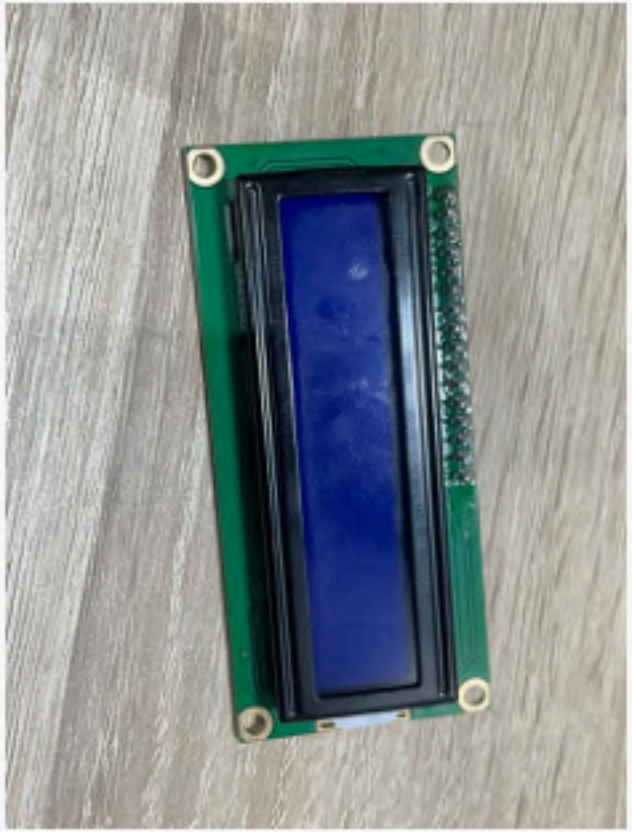
Expérience Réelle



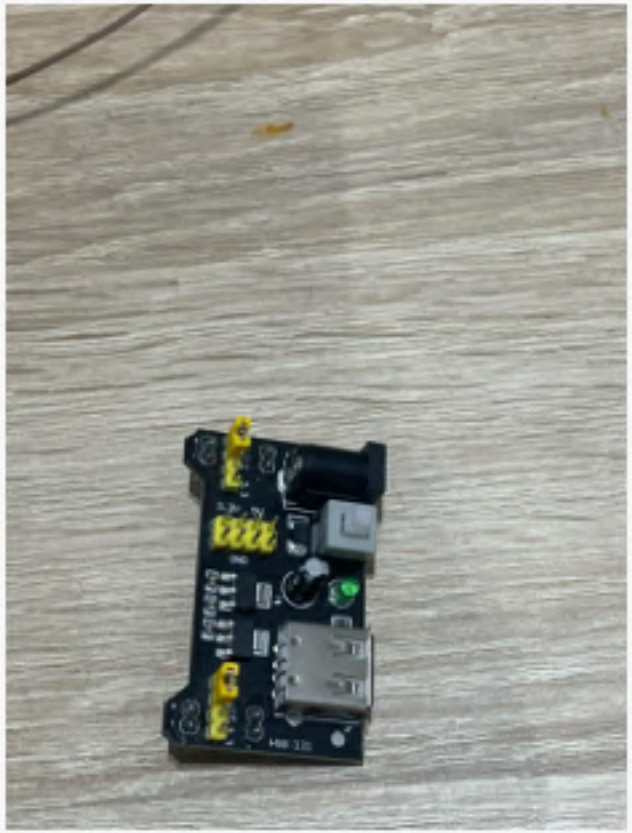
Composition du Système



Carte Arduino UNO (5V)



Écran LCD 16x2

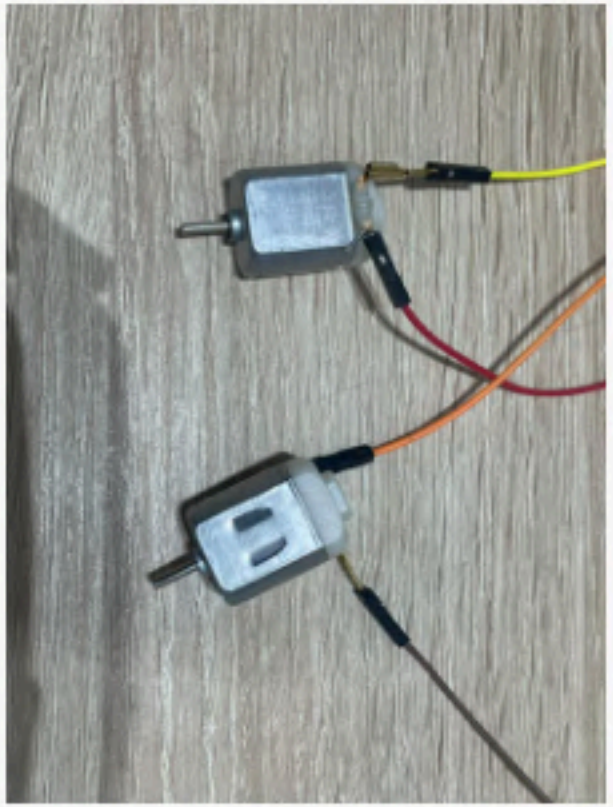


Carte d'alimentation (5V)

Composition du Système



2 Moteurs DC



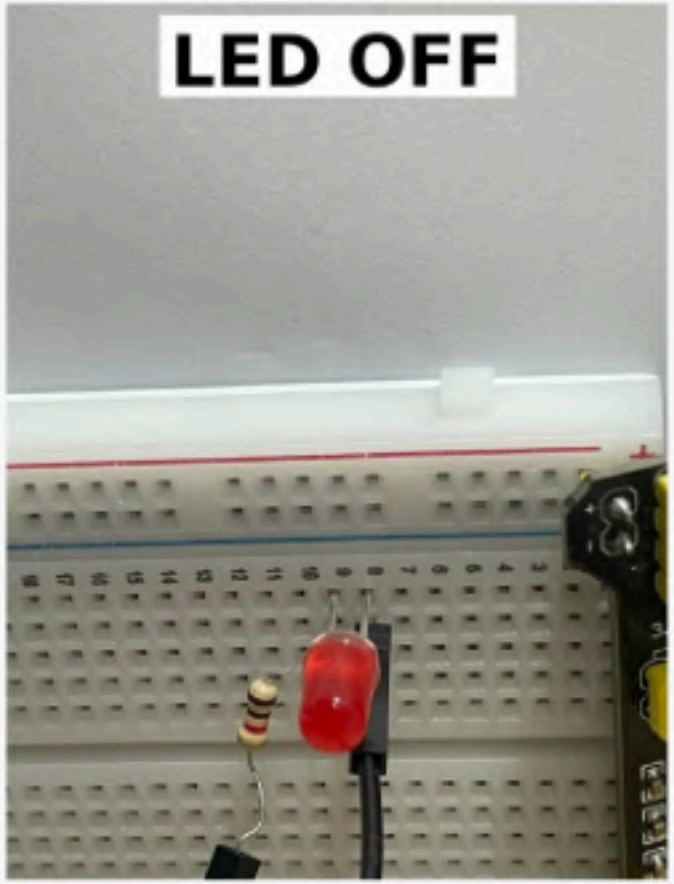
LED & Buzzer



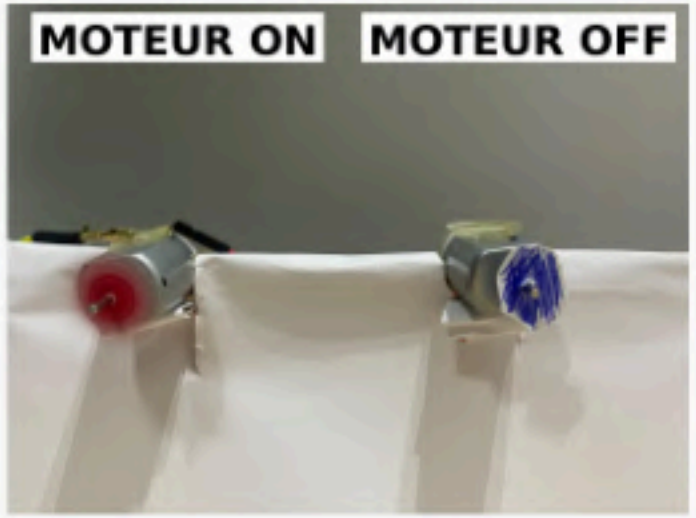
Capteur de température

Expérience Réelle — Conditions Imposées

Tc = 30°C | Tmax = 35°C



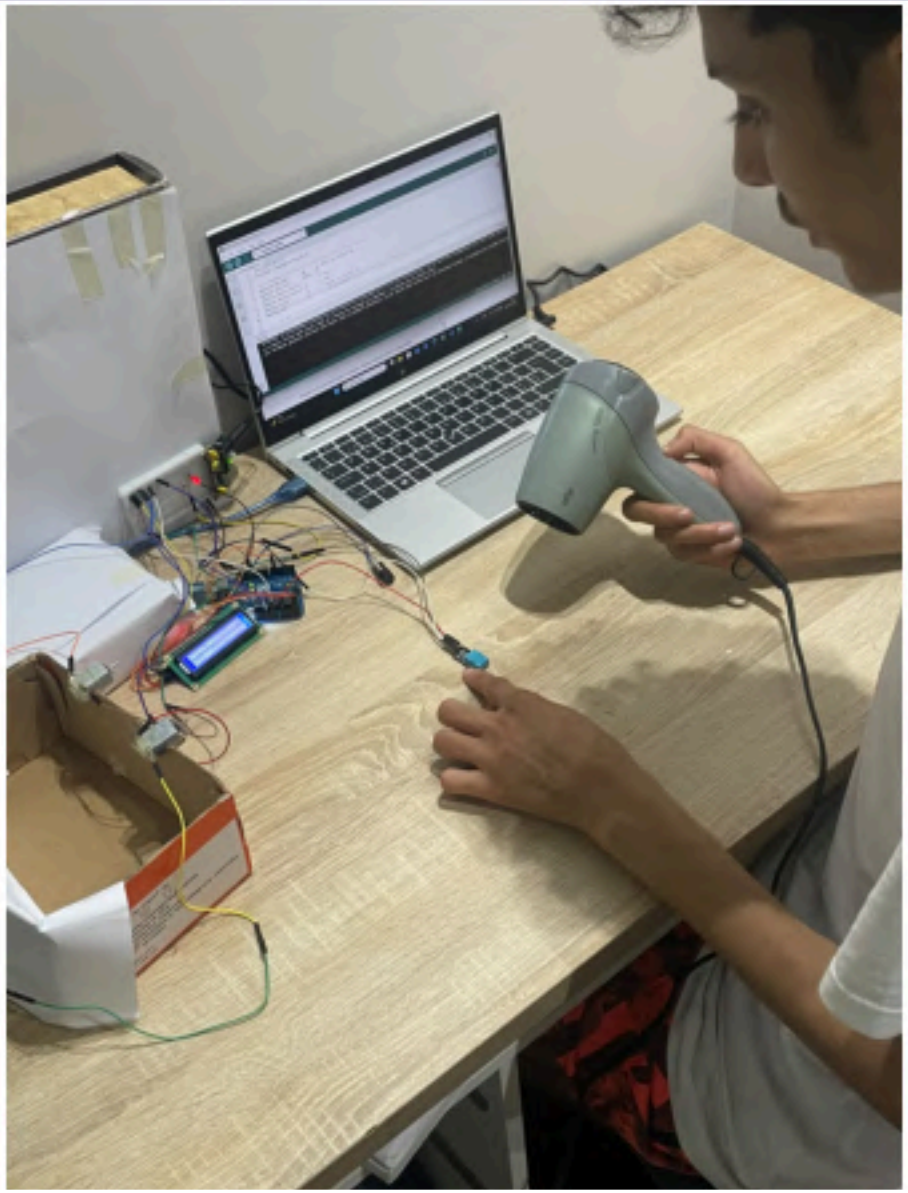




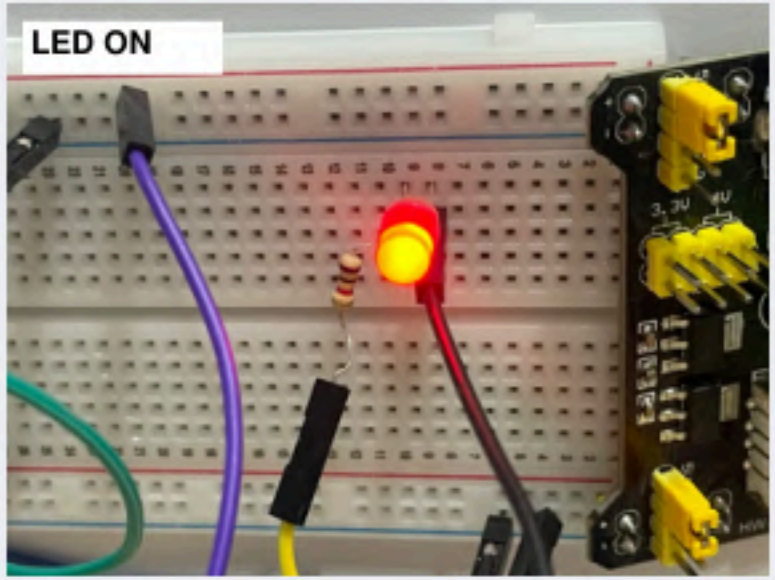
Expérience Réelle

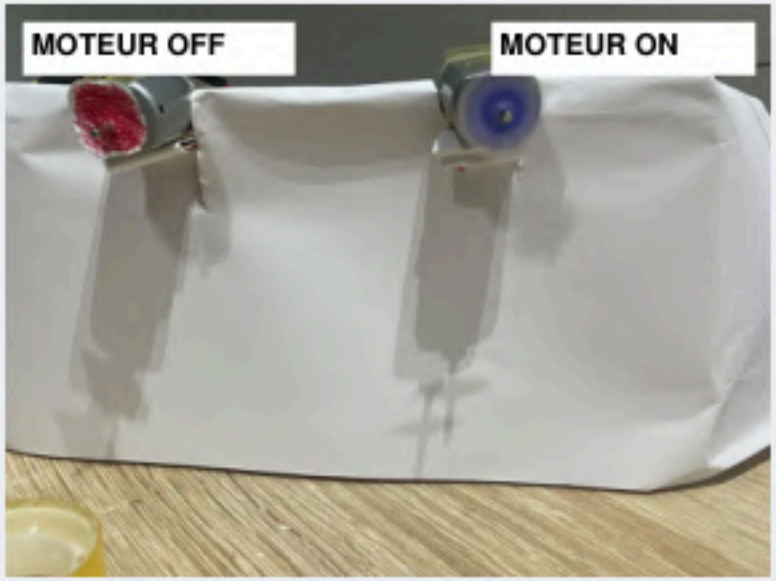
Réchauffement pour

augmenter la température



Expérience Réelle







Cas 1 : $T < Tc$

- LED OFF, Buzzer OFF
- Moteur de réchauffement ON → chauffe pour ramener T vers Tc
- Moteur de refroidissement OFF

Cas 2 : $T > Tmax > Tc$ (surchauffe)

- LED rouge ON + Buzzer ON → alarme déclenchée
- Moteur de réchauffement OFF
- Moteur de refroidissement ON → refroidit pour revenir à Tc

Avantages

- Sécurité : alarme automatique (LED + Buzzer) en cas de surchauffe
- Boucle fermée : le système essaie en permanence d'atteindre Tc

Contraintes

- Les moteurs 5V ne peuvent pas modifier la température efficacement
  vu leur faible alimentation

1. 1er principe de la thermodynamique :

La variation d'énergie interne de l'enceinte vaut :

$$\frac{dU}{dt} = P_{fournie}(t) - P_{perdue}(t) \quad \Rightarrow \quad C_{th} \cdot \frac{dT_{mes}}{dt} = P_{fournie}(t) - P_{perdue}(t)$$

2. Loi de Fourier — flux thermique perdu :

Le flux thermique traversant la surface S de l'enceinte vers l'extérieur :

$$J_{th} = \frac{T_{mes}(t) - T_{amb}}{R_{th}} \quad \Rightarrow \quad P_{perdue}(t) = J_{th} \cdot S = \frac{T_{mes}(t) - T_{amb}}{R_{th}} \cdot S$$

3. En substituant dans le bilan :

$$C_{th} \cdot \frac{dT_{mes}}{dt} = P_{fournie}(t) - \frac{T_{mes}(t) - T_{amb}}{R_{th}} \cdot S$$

4. Solution générale (système 1er ordre) :

C'est une équation différentielle linéaire du 1er ordre. Avec $\tau = R_th \times C_th \times S$:

$$T_{mes}(t) = (T_{mes}(0) - T_\infty) \cdot e^{-t/\tau} + T_\infty \qquad \tau = R_{th} \cdot C_{th} \cdot S$$

5. Déduction de la stabilité :

Quand $t \to +\infty$, le terme transitoire disparaît :

$$\lim_{t \to +\infty} (T_{mes}(0) - T_\infty) \cdot e^{-t/\tau} = 0 \quad \Rightarrow \quad T_{mes}(t) \longrightarrow T_\infty$$

Condition de stabilité :

$$\tau > 0 \implies R_{th} > 0 \text{ et } C_{th} > 0$$

6. Expression de P(t) à fournir à l'actionneur :

On isole P(t) depuis l'équation différentielle :

$$P(t) = C_{th} \cdot \frac{dT_{mes}}{dt} + \frac{T_{mes}(t) - T_{amb}}{R_{th}} \cdot S$$

7. A l'état stationnaire (dT/dt = 0 , T_mes = T_consigne) :

Le terme transitoire disparaît. La puissance nécessaire pour maintenir la consigne :

$$P = \frac{T_{consigne} - T_{amb}}{R_{th}} \cdot S$$

Rappel : P à l'état stationnaire :

$$P = \frac{T_{consigne} - T_{amb}}{R_{th}} \cdot S$$

Comment minimiser P ?

Le terme à réduire est : (T_consigne – T_amb) / R_th × S

- **Augmenter R_th :**

  Mieux isoler l'enceinte → les pertes thermiques diminuent → P ↓

- **Diminuer S :**

  Réduire la surface d'échange avec l'extérieur → moins de pertes → P ↓

$$\frac{S}{R_{th}} \downarrow \implies P \downarrow \Rightarrow \text{láctionneur est moins sollicité}$$

La modélisation thermique confirme que le régulateur PID assure une régulation efficace de la température ambiante.

Merci pour votre attention !

vout.TEMP.PY > ...

```python
import math
import matplotlib.pyplot as plt

R0 = 10000
B = 3950
T0 = 298.15

RCTN = []
TEMP = []
for i in range(-20, 60):
    T = i + 273.15

    TEMP.append(i)   # température en °C pour affichage

    R = R0 * math.exp(B * (1/T - 1/T0))

    RCTN.append(R)

plt.plot(TEMP, RCTN)

plt.xlabel("Temperature (°C)")
plt.ylabel("Resistance (Ohms)")
plt.title("CTN")
plt.grid()
plt.show()
```

$R_{CTN}(T)$ — *Courbe caractéristique de la CTN*

```python
import matplotlib.pyplot as plt
import numpy as np

# Paramètres
Vcc = 5.0  # Tension d'alimentation (V)
RCTN_min = 1.0   # kΩ (à ~100°C)
RCTN_max = 32.0  # kΩ (à ~0°C)

# Valeurs de Rfixe à tester
curves = [
    (2,  'red',    'R_fixe = 2 kΩ'),
    (5,  'orange', 'R_fixe = 5 kΩ'),
    (10, 'green',  'R_fixe = 10 kΩ  ← optimal'),
    (20, 'blue',   'R_fixe = 20 kΩ'),
    (50, 'purple', 'R_fixe = 50 kΩ'),
]

# Axe R_CTN
RCTN = np.linspace(RCTN_min, RCTN_max, 500)

# Tracé
fig, ax = plt.subplots(figsize=(10, 6))
fig.patch.set_facecolor('#E6E8F8')
ax.set_facecolor('#F8F9FF')

```

```
Click to add a breakpoint or, label in curves:
        Vout = Vcc * Rfixe / (Rfixe + RCTN)
        lw = 3 if Rfixe == 10 else 1.5
        ax.plot(RCTN, Vout, color=color, linewidth=lw, label=label)

    # Point optimal : Rfixe = 10, RCTN = 10 → Vout = 2.5V
    ax.plot(10, 2.5, 'o', color='green', markersize=10, zorder=5)
    ax.annotate('R_fixe = R₀ = 10 kΩ\nV_out = 2,5 V',
                xy=(10, 2.5), xytext=(14, 2.1),
                fontsize=10, color='green',
                arrowprops=dict(arrowstyle='->', color='green'))

    # Ligne horizontale à 2.5V
    ax.axhline(y=2.5, color='gray', linestyle='--', linewidth=1, alpha=0.7)

    # Mise en forme
    ax.set_xlabel('R_CTN (kΩ)          ← T élevée (100°C) ..... T basse (0°C) →',
                  fontsize=11, color='#323278')
    ax.set_ylabel('V_out (V)', fontsize=11, color='#323278')
    ax.set_title('V_out en fonction de R_CTN pour différentes valeurs de R_fixe\n'
                 r'$V_{out} = V_{cc} \times \frac{R_{fixe}}{R_{fixe} + R_{CTN}}$',
                 fontsize=13, color='#323278')

    ax.set_xlim(RCTN_min, RCTN_max)
    ax.set_ylim(0, 5)
```

```
51    ax.set_yticks([0, 1, 2, 2.5, 3, 4, 5])
52    ax.set_xticks([1, 5, 10, 15, 20, 25, 30])
53    ax.grid(True, linestyle='--', alpha=0.5)
54    ax.legend(loc='upper right', fontsize=10)
55
56    plt.tight_layout()
57    plt.savefig('graphe_vout_rctn.png', dpi=150)
58    plt.show()
59    print("Graphe sauvegardé : graphe_vout_rctn.png")
```

Vout en fonction de Rctn pour différentes valeurs de Rfixe

```
#include <math.h>
#include <LiquidCrystal_I2C.h>

// ── Broches ──────────────────────────────────────────────
#define capteur           A0
#define moteur_chauffe    5
#define moteur_refroid    6
#define buzzer            13
#define led               12

// ── LCD ──────────────────────────────────────────────────
LiquidCrystal_I2C lcd(32, 16, 2);

// ── Variables PID ────────────────────────────────────────
double erreur;
double temperature;
double derniere_temp;
double kP = 1.0, kI = 0.0, kD = 0.0;
double P, I, D;
double PID;
float dt;
long dernier_temps;
double consigne = 50.0;
int commande_ideale = 0;
int commande = 0;
```

```
27    // ── Variables optimisation ──────────────────────────
28    float energie   = 0;
29    float t_stable  = 0;
30    bool  stabilise = false;
31    unsigned long t_debut = 0;
32
33    // ── Seuils alarme ───────────────────────────────────
34    float T_MAX = 70.0;
35    float T_MIN = 0.0;
36
37    // ── Affichage LCD ───────────────────────────────────
38    void afficherLCD() {
39      lcd.clear();
40      lcd.setCursor(0, 0);
41      lcd.print("Tm:");
42      lcd.print(temperature, 1);
43      lcd.print("C");
44      lcd.setCursor(0, 1);
45      lcd.print("Tc:");
46      lcd.print(consigne, 0);
47      lcd.print("C");
48      if (stabilise) {
49        lcd.print(" tr:");
50        lcd.print(t_stable / 1000.0, 1);
```

```
52      }
53    }
54    // ── Alarme ──────────────────────────────────────────────────────
55    void gererAlarme() {
56      if (temperature > T_MAX || temperature < T_MIN) {
57        digitalWrite(led, HIGH);
58        tone(buzzer, 500, 375);
59        delay(375);
60        digitalWrite(led, LOW);
61        delay(375);
62      } else {
63        digitalWrite(led, LOW);
64        noTone(buzzer);
65      }
66    }
67
68    // ═══════════════════════════════════════════════════════════════
69    void setup() {
70      Serial.begin(9600);
71      lcd.begin(16, 2);
72      lcd.init();
73      lcd.backlight();
```

```
75      lcd.setCursor(2, 0);
76      lcd.print("TIPE PID");
77      lcd.setCursor(2, 1);
78      lcd.print("Demarrage...");
79      delay(2000);
80      lcd.clear();
81      pinMode(capteur,          INPUT);
82      pinMode(moteur_chauffe,  OUTPUT);
83      pinMode(moteur_refroid,  OUTPUT);
84      pinMode(buzzer,          OUTPUT);
85      pinMode(led,             OUTPUT);
86      t_debut        = millis();
87      dernier_temps  = millis();
88  }
89
90  // ================================================================
91  void loop() {
92
93      // 1. Lecture température
94      temperature = (analogRead(capteur) * (5000.0 / 1024.0) - 500) / 10;
95
96      // 2. Calcul de l erreur
97      erreur        = consigne - temperature;
```

```
99    dernier_temps = millis();
100
101   // 3. Calcul PID
102   P = erreur * kP;
103   I = I + (erreur * kI) * dt;
104   D = (derniere_temp - temperature) * kD / dt;
105   derniere_temp = temperature;
106   PID = P + I + D;
107
108   // 4. Commande moteurs
109   commande = PID + commande_ideale;
110
111   if (commande < 0) {
112     commande = sqrt((PID + commande_ideale) * (PID + commande_ideale));
113     analogWrite(moteur_refroid, commande);
114     analogWrite(moteur_chauffe, commande_ideale);
115   } else {
116     analogWrite(moteur_chauffe, commande);
117     analogWrite(moteur_refroid, commande_ideale);
118   }
119
120   // 5. Mesure energie
121   energie += abs(commande) * 0.01;
122
```

```
124   if (!stabilise && abs(erreur) < consigne * 0.05) {
125     stabilise = true;
126     t_stable  = millis() - t_debut;
127   }
128
129   // 7. Affichage LCD
130   afficherLCD();
131
132   // 8. Alarme buzzer + LED
133   gererAlarme();
134
135   // 9. Envoi donnees Serial
136   Serial.print("Temperature:"); Serial.print(temperature);
137   Serial.print(" PID:");         Serial.print(PID);
138   Serial.print(" Commande:");    Serial.print(commande);
139   Serial.print(" Energie:");     Serial.println(energie);
140
```

```
#include <DHT.h>
#include <LiquidCrystal_I2C.h>

// ┤ Broches ├
#define DHT_PIN          A0
#define DHT_TYPE         DHT11
#define moteur_chauffe   5
#define moteur_refroid   3
#define buzzer           13
#define led              6

// ┤ LCD I2C ├
LiquidCrystal_I2C lcd(0x27, 16, 2);

// ┤ DHT11 ├
DHT dht(DHT_PIN, DHT_TYPE);

// ┤ Variables PID ├
double erreur;
double temperature     = 25.0;
double derniere_temp   = 25.0;
double kP = 2.0, kI = 0.1, kD = 0.5;
double P, I_term, D;
double PID;
float  dt;
long   dernier_temps;
```

```ino
#include <DHT.h>
#include <LiquidCrystal_I2C.h>

// ── Broches ──────────────────────────────────────────────
#define DHT_PIN          A0       // DHT11 sur broche A0
#define DHT_TYPE         DHT11
#define moteur_chauffe   5        // PWM ✓
#define moteur_refroid   3        // PWM ✓ (changé depuis 6, occupé par LED)
#define buzzer           13       // Buzzer sur broche 13
#define led              6        // LED sur broche 6

// ── LCD I2C ──────────────────────────────────────────────
LiquidCrystal_I2C lcd(0x27, 16, 2);

// ── DHT11 ────────────────────────────────────────────────
DHT dht(DHT_PIN, DHT_TYPE);

// ── Variables PID ────────────────────────────────────────
double erreur;
double temperature     = 25.0;  // valeur initiale
double derniere_temp   = 25.0;
double kP = 2.0, kI = 0.1, kD = 0.5;
double P, I_term, D;
double PID;
float dt;
```

```ino
#include <DHT.h>
#include <LiquidCrystal_I2C.h>

// ── Broches ──────────────────────────────────────────────
#define DHT_PIN          A0       // DHT11 sur broche A0
#define DHT_TYPE         DHT11
#define moteur_chauffe   5        // PWM ✓
#define moteur_refroid   3        // PWM ✓ (changé depuis 6, occupé par LED)
#define buzzer           13       // Buzzer sur broche 13
#define led              6        // LED sur broche 6

// ── LCD I2C ──────────────────────────────────────────────
LiquidCrystal_I2C lcd(0x27, 16, 2);

// ── DHT11 ────────────────────────────────────────────────
DHT dht(DHT_PIN, DHT_TYPE);

// ── Variables PID ────────────────────────────────────────
double erreur;
double temperature     = 25.0;  // valeur initiale
double derniere_temp   = 25.0;
double kP = 2.0, kI = 0.1, kD = 0.5;
double P, I_term, D;
double PID;
```

```
26  long   dernier_temps;
27  double consigne = 30.0;
28  int    commande_ideale = 0;
29  int    commande = 0;
30
31  // ── Variables optimisation ──────────────────────────────
32  float         energie   = 0;
33  float         t_stable  = 0;
34  bool          stabilise = false;
35  unsigned long t_debut   = 0;
36
37  // ── Seuils alarme ───────────────────────────────────────
38  float T_MAX = 35.0;
39  float T_MIN = 5.0;
40
41  // ── Timers non-bloquants ────────────────────────────────
42  unsigned long dernierAffichage = 0;
43  unsigned long derniereDHT      = 0;
44  #define INTERVALLE_LCD 200    // affichage toutes les 200ms
45  #define INTERVALLE_DHT 1000   // lecture DHT toutes les 1s
46
47  // ── Affichage LCD ───────────────────────────────────────
48  void afficherLCD() {
49    lcd.setCursor(0, 0);
```

```
51      lcd.print(temperature, 1);
52      lcd.print("C    ");
53
54      lcd.setCursor(0, 1);
55      lcd.print("Tc:");
56      lcd.print(consigne, 0);
57      lcd.print("C         ");
58    }
59
60    // ── Alarme non-bloquante ──────────────────────────────
61    void gererAlarme() {
62      if (temperature > T_MAX || temperature < T_MIN) {
63        digitalWrite(led, HIGH);
64        tone(buzzer, 500);
65      } else {
66        digitalWrite(led, LOW);
67        noTone(buzzer);
68      }
69    }
70
71    // ── Limiter entre 0 et 255 ────────────────────────────
72    int clamp(int val) {
73      if (val <= 0)  return 0;
74      if (val < 100) return 100;   // vitesse minimale demarrage
```

```
75      if (val > 180) return 180;  // vitesse maximale
76      return val;
77    }
78
79    // ============================================================
80    void setup() {
81      Serial.begin(9600);
82
83      lcd.init();
84      lcd.backlight();
85      lcd.clear();
86      lcd.setCursor(2, 0);
87      lcd.print("TIPE  PID");
88      lcd.setCursor(2, 1);
89      lcd.print("Demarrage...");
90      delay(2000);
91      lcd.clear();
92
93      dht.begin();
94
95      pinMode(moteur_chauffe, OUTPUT);
96      pinMode(moteur_refroid, OUTPUT);
97      pinMode(buzzer,         OUTPUT);
98      pinMode(led,            OUTPUT);
```

```
99
100   analogWrite(moteur_chauffe, 0);
101   analogWrite(moteur_refroid, 0);
102
103   t_debut        = millis();
104   dernier_temps = millis();
105   I_term         = 0;
106
107   Serial.println("Systeme PID demarre");
108 }
109
110 // ============================================================
111 void loop() {
112
113   unsigned long maintenant = millis();
114
115   // 1. Lecture DHT11 toutes les 1 seconde (non-bloquant)
116   if (maintenant - dernieredHT >= INTERVALLE_DHT) {
117     dernieredHT = maintenant;
118     float lecture = dht.readTemperature();
119     if (!isnan(lecture)) {
120       temperature = lecture;
121     } else {
```

```
123     }
124   }
125
126   // 2. Calcul dt et erreur
127   dt = (maintenant - dernier_temps) / 1000.0;
128   if (dt <= 0) dt = 0.01;
129   dernier_temps = maintenant;
130
131   erreur = consigne - temperature;
132
133   // 3. Calcul PID
134   P      = erreur * kP;
135   I_term = I_term + (erreur * kI) * dt;
136   I_term = constrain(I_term, -255, 255);
137   D      = (derniere_temp - temperature) * kD / dt;
138   derniere_temp = temperature;
139   PID = P + I_term + D;
140
141   // 4. Commande moteurs
142   commande = (int)(PID + commande_ideale);
143
144   if (commande < 0) {
145     analogWrite(moteur_refroid, clamp(-commande));
```

```
146       analogWrite(moteur_chauffe, 0);
147     } else {
148       analogWrite(moteur_chauffe, clamp(commande));
149       analogWrite(moteur_refroid, 0);
150     }
151
152     // 5. Mesure energie
153     energie += abs(commande) * dt * 0.01;
154
155     // 6. Detection stabilisation
156     if (!stabilise && abs(erreur) < consigne * 0.05) {
157       stabilise = true;
158       t_stable  = millis() - t_debut;
159     }
160
161     // 7. Affichage LCD toutes les 200ms
162     if (millis() - dernierAffichage >= INTERVALLE_LCD) {
163       afficherLCD();
164       dernierAffichage = millis();
165     }
166
167     // 8. Alarme (non-bloquante)
168     gererAlarme();
```

```
169
170     // 9. Serial
171     Serial.print("Temp:");  Serial.print(temperature, 2);
172     Serial.print(" Err:");  Serial.print(erreur, 2);
173     Serial.print(" PID:");  Serial.print(PID, 2);
174     Serial.print(" Cmd:");  Serial.println(commande);
175 }
176
```